

```

→ os git:(main) × ./b
Enter number of processes: 3
Enter number of resources: 3
Enter total available resources of each type:
Resource 0: 1
Resource 1: 2
Resource 2: 1
Enter maximum demand matrix (max) for each process:
Maximum demand for process P0 (space separated for each resource): 3 3 0
Maximum demand for process P1 (space separated for each resource): 2 3 2
Maximum demand for process P2 (space separated for each resource): 3 2 0
Enter allocation matrix for each process:
A → os git:(main) × gcc memoryAllocationTechniques.c -o m
A → os git:(main) × ./m
AEnter number of memory blocks: 3
$Enter block sizes:
$1 2 0
EEnter number of processes: 3
EEnter process sizes:
E1 2 1

```

First Fit Allocation:

Process No.	Process Size	Block No.
1	1	1
2	2	2
3	1	Not Allocated

Best Fit Allocation:

Process No.	Process Size	Block No.
1	1	1
2	2	2
3	1	Not Allocated

Worst Fit Allocation:

Process No.	Process Size	Block No.
1	1	2
2	2	Not Allocated
3	1	1

```

→ PageRplcAlgo git:(main) × ./f

```

```

Enter number of pages: 6
Enter the reference string: 2 1 6 0 2 1
Enter number of frames: 3

```

Page Reference String Processing (FIFO):

```

Page 2 -> 2 - - (Page Fault)
Page 1 -> 2 1 - (Page Fault)
Page 6 -> 2 1 6 (Page Fault)
Page 0 -> 0 1 6 (Page Fault)
Page 2 -> 0 2 6 (Page Fault)
Page 1 -> 0 2 1 (Page Fault)

```

Total Page Faults = 6

```
→ PageRplcAlgo git:(main) × ./l
Enter number of pages: 6
Enter the reference string: 2 4 1 0 6 7
Enter number of frames: 3
```

```
Page Reference String Processing (LRU):
Page 2 -> 2 - - (Page Fault)
Page 4 -> 2 4 - (Page Fault)
Page 1 -> 2 4 1 (Page Fault)
Page 0 -> 0 4 1 (Page Fault)
```

```
→ PageRplcAlgo git:(main) × ./op
Enter number of pages: 6
Enter the reference string: 1 3 2 2 4 6
Enter number of frames: 3
```

```
Page Reference String Processing (Optimal):
Page 1 -> 1 - - (Page Fault)
Page 3 -> 1 3 - (Page Fault)
Page 2 -> 1 3 2 (Page Fault)
Page 2 -> 1 3 2
Page 4 -> 4 3 2 (Page Fault)
Page 6 -> 6 3 2 (Page Fault)
```

Total Page Faults = 5

```
→ PageRplcAlgo git:(main) × ./p
Enter number of pages: 7
Enter the reference string: 1 2 1 3 8 1
3
Enter number of frames: 3
```

```
Page Reference String Processing (Second Chance):
Page 1 -> 1(0) - - (Page Fault)
Page 2 -> 1(0) 2(0) - (Page Fault)
Page 1 -> 1(1) 2(0) -
Page 3 -> 1(1) 2(0) 3(0) (Page Fault)
Page 8 -> 1(0) 8(0) 3(0) (Page Fault)
Page 1 -> 1(1) 8(0) 3(0)
Page 3 -> 1(1) 8(0) 3(1)
```

Total Page Faults = 4

```

→ DiskScheduling git:(main) × ./g
Enter the number of requests: 5
Enter the disk request queue: 1 5 1 2 3
Enter the starting position of the disk head: 1
Head movement sequence: 1 -> 1 -> 5 -> 1 -> 2 -> 3
→ DiskScheduling git:(main) × ./s

```

```

Enter total number of requests: 6
Enter initial position of disk head: 1
Enter elements of disk request queue: 2 1 3 2 1 5

Head movement sequence: 1 -> 1 -> 1 -> 2 -> 2 -> 3 -> 5
Total Head Movement (Seek Time): 4

```

```

→ DiskScheduling git:(main) × ./y
Enter number of requests: 6
Enter requests: 2 1 3 2 1 6
Enter the head position: 2
Enter the direction (left/right): left

```

```

Head movement sequence:
2 -> 1 -> 1 -> 0 -> 3 -> 6

```

```

→ DiskScheduling git:(main) × ./cc
Enter number of requests: 5
Enter requests: 1 2 1 3 2
Enter the head position: 1

Seek Sequence: 2 2 3 199 0 0
Total Head Movement (cylinders) = 397
Total Seek Time = 1985 ms

```

```

→ DiskScheduling git:(main) × gcc look.c -o l
→ DiskScheduling git:(main) × ./l
Enter number of requests: 6
Enter the request queue: 1 2 1 3 2 6
Enter the initial head position: 3
Enter the initial direction (left/right): right

Seek Sequence: 6 2 2 1 1
Total Head Movement (cylinders) = 8
Total Seek Time = 40 ms

```

→ DiskScheduling git:(main) × gcc c-look.c -o cl

→ DiskScheduling git:(main) × ./cl

Enter number of requests: 6

Enter the request queue: 2 1 3 1 4 5

Enter the initial head position: 2

Enter the direction of head movement (left/right): left

Seek Sequence: 1 1 3 4 5

Total Head Movement (cylinders) = 5

Total Seek Time = 25 ms

—